

SAP OData API Integration for Synthetic Data Generation

This document explains the structure and purpose of the OData APIs utilized in the TDE (Test Data Engine) synthetic data generation process. Our solution relies on two distinct OData services communicating with the SAP S/4HANA backend to complete the "fetch-synthesize-insert" lifecycle.

1. Reference Data API: API_SALES_ORDER_SRV

Endpoint Path: /sap/opu/odata/sap/API_SALES_ORDER_SRV/

Type: Standard SAP OData V2 API

What it does:

This is SAP's standard API for managing Sales Orders. In our application, we use it strictly for **Reading** and **Profiling** rather than writing.

1. **Fetching Reference Rows:** We query this API (specifically the `A_SalesOrder` entity) to retrieve a real sales document (e.g., Sales Order 2). This real document acts as the "seed" or reference for the Python ML Engine.
2. **Deriving Maximum IDs:** We query this API (`$top=1&$orderby=SalesOrder desc`) to find the highest existing 99* sales order number currently in the system, which allows the engine to strictly increment the next generated synthetic order without primary key collisions.

Key Entity Used:

- **A_SalesOrder:** Contains the root sales document header.
-

2. Synthetic Data Insertion API: FDE/TDE_GEN_SRV

Endpoint Path: /sap/opu/odata/FDE/TDE_GEN_SRV/

Type: Custom ABAP OData Service (Z-Service)

What it does:

Because standard SAP APIs often have complex business validation rules that reject partial or artificially generated data, we use this custom Z-Service to **bypass standard BAPIs** and insert the synthetic data directly into the database tables (Type A, B, and C).

Key Entities Used:

FetchData

- **Purpose:** Used for fetching structural definitions or raw records directly from arbitrary ABAP tables.
- **Properties:**
 - `RequestId`: String (Key)
 - `PAYLOAD`: String (JSON)

InsertData (InsertDataSet)

- **Purpose:** This is the core entity used to write the synthesized records into the SAP database.
- **Properties:**
 - `RequestId`: String (e.g., "INSERT_001")
 - `Payload`: String (A stringified JSON payload containing the table name, the operation mode, and the row data)

The Insert Payload Structure

To successfully insert a table, the backend Node.js server sends a POST request to `InsertDataSet` with a stringified `Payload` field that looks like this:

```
{
  "table": "VBAK",
  "mode": "I",
  "rows": [
    {
      "mandt": "100",
      "vbeln": "9962108432",
      "erdat": "2017-10-06",
      "netwr": 175.5
      // ... other synthesized fields
    }
  ]
}
```

- **table**: The target SAP database table (e.g., VBAK, VBAP, VBPA, VBKD).
- **mode**: Represents the database operation. Set to "I" for **Insert**.
- **rows**: An array containing the individual record objects (with remapped 99* primary keys and synthesized data).

Lifecycle Summary

1. **Fetch**: API_SALES_ORDER_SRV pulls a reference order.
2. **Synthesize**: The Python ML Engine generates new data and assigns a new incremental 99* key.
3. **Insert**: The Node.js server packages the new data into the InsertData JSON format and POSTs it to FDE/TDE_GEN_SRV. The ABAP backend parses the JSON and writes the records directly into the corresponding tables (VBAK, VBAP, etc.), seamlessly making the synthetic order available in SAP Logon (VA03).